

Fairness-Centric Global Placement

Min-Max Placement via Howard's Algorithm and Bipartite Matching

Wai-Shing Luk

luk036.github.io

2026

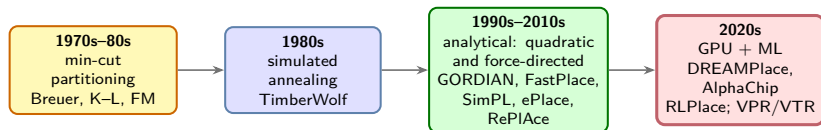
Outline

- 1 Background
- 2 Placement
- 3 Objective
- 4 Howard Algorithm
- 5 Legalization and I/O
- 6 The Placer
- 7 Congestion and Results
- 8 Routing and Conclusion

Section 1

Background

A Four-Decade History of Placement



- Four waves, one constant: each optimized an **aggregate** – total cut, total wirelength, total overflow.

Wirelength Models

- **Quadratic**: sum of squared distances $\Rightarrow$ sparse linear system (GORDIAN, NTUplace3).
 - Mismatches Manhattan routing and the **concave** wire costs of FPGA fabrics.
 - Needs pseudo-nets / pseudo-IO $\Rightarrow$ *accidental* complexity.
- **HPWL**: bounding-box half-perimeter; a tight Manhattan lower bound.
 - Piecewise linear $\Rightarrow$ **non-differentiable** where two pins tie for an edge.
- **Smooth surrogates** – log-sum-exp (LSE) and weighted average (WA):
 - Used by FastPlace, SimPL, ePlace, RePIAce.
 - Still a **sum over nets** $\Rightarrow$ blind to *where* the demand is spent.

Section 2

Placement

Why Global Placement Is Hard

- **Placement** assigns every module a grid site: NP-hard combinatorial optimization.
- It must respect **no-overlap**, grid boundaries, and reserved columns (DSP/SRAM).
- True routed delay cannot be optimized directly, so a **wire-length proxy** is used.
- Moving one module changes *all* of its nets: modules are coupled through the netlist and the occupancy grid.

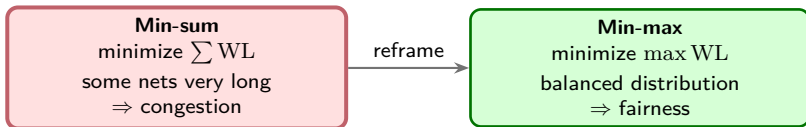
The HPWL Proxy and Its Blind Spot

$$\text{HPWL}(e) = \left(\max_{i \in e} x_i - \min_{i \in e} x_i \right) + \left(\max_{i \in e} y_i - \min_{i \in e} y_i \right)$$

- Convex but **non-smooth**.
- **Underestimates** routing for multi-pin nets ($n > 3$).
- Minimizing the **sum** creates blind spots: a few long nets are sacrificed to improve the average.
- The long nets cause timing violations and routing congestion.

Fairness, Not Scarcity

- Congestion is usually not a shortage of routing resources but an **uneven distribution** of demand.
- “We do not worry about scarcity, but about unfairness.”
- Change the objective: minimize the **worst** connection, not the total.



Section 3

Objective

Minimize the Worst Wire Length

$$\min_{\text{placement}} \max_{(u,v) \in E} (c_x |x_u - x_v| + c_y |y_u - y_v|)$$

- A **bottleneck** objective: it bounds *every* connection.
- Convex (a maximum of linear terms) yet **non-smooth**.
- The linear terms may be replaced by any **monotone** (even concave) cost $m(\cdot)$, matching the concave wire costs of real FPGA fabrics.

Fairness Is Not Equality

$$\min_{\text{placement}} \max_{e \in E} \frac{\text{worst}(e)}{\omega_e}$$

- Weight net e by ω_e : its timing criticality, or switching activity for power.
- A critical net may spend a larger wire budget before it binds the objective.
- The weights enter the **per-arc costs** only – Howard's algorithm is unchanged.
- Weighted max-min fairness **reconciles** fairness with timing/power instead of trading them off.
- A lexicographic variant (critical nets first, then equalize the rest) is a further option.

One Axis at a Time

- Optimize x with y frozen, then y with x frozen (alternating directions).
- Let q_v be the coordinate on the axis under optimization and r a trial wire-length **radius**:

$$q_v - q_u \leq \text{cost}((u, v), r), \quad \forall (u, v) \in E.$$

- These are **difference constraints**: feasible *iff* the constraint graph has **no negative cycle**.
- Increasing r relaxes the constraints, so the smallest feasible r is set by the tightest cycle.

The Minimum Cycle Ratio

- A negative cycle of length n with total cost C forces $n r \geq C$, i.e. $r \geq C/n$.
- Hence the smallest feasible radius is the **minimum cycle ratio** (maximum cycle mean):

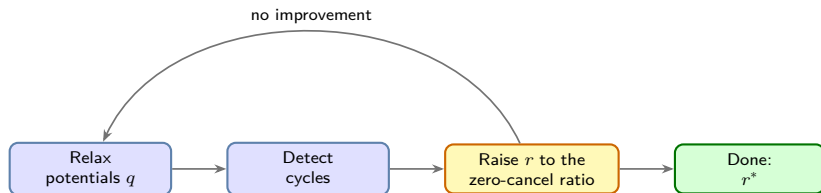
$$r^* = \max_{\text{cycles } \kappa} \frac{\sum_{(u,v) \in \kappa} c_{uv}}{|\kappa|}.$$

- Worst-wire-length placement reduces exactly to a **parametric minimum-cost-flow** problem. 

Section 4

Howard Algorithm

Policy Iteration for the Minimum Cycle Ratio



- Bellman-Ford-style relaxation, cycle detection in the policy graph, then raise r to the critical cycle's zero-cancel ratio; alternate the search direction.

The Parametric Formulation

- Tighten the constraints by the radius:

$$q_v - q_u \leq \text{cost}(u, v) - r \tau_{uv}.$$

- The **zero-cancel** ratio of a cycle κ is $\sum c_{uv}/|\kappa|$: the r that makes the cycle's total weight zero.
- As r rises, negative cycles disappear; the final r is the **worst wire-length budget** on that axis.

Integer-Only Distance

With $r = n/d$ and an integer arc cost c :

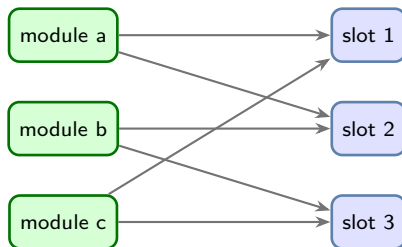
$$\text{dist}(e, r) = \left\lfloor \frac{n - c d}{\delta d} \right\rfloor.$$

- One integer floor-division per arc: the whole sweep avoids floating point.
- Exact rationals near integer boundaries, with a fast integer inner loop.
- Engine: `digraphx.MinParametricSolver` and the negative-cycle finder.

Section 5

Legalization and I/O

Legalization by Bipartite Matching



- Bucket modules that share a coordinate; candidates are the reachable sites in a growing window.
- The edge weight is the **change in worst wire length** if the module moves there:

$$\min_{\mu} \sum_v \Delta_{\text{worst}}(v \rightarrow \mu(v)).$$

- A local window is tried first; a **global fallback** offers every free site on the line

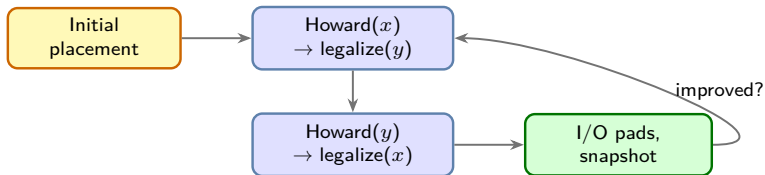
I/O Pad Ring Assignment

- Pads must lie on the **border ring** around the core.
- Pick the side and coordinate nearest the pad's connected modules, respecting per-side capacity.
- Spread pads along each edge with the same matching machinery used for cells.
- Pads and cells **share the per-line capacity**, so any cap must be applied consistently.

Section 6

The Placer

The Outer Optimization Loop



- A round is accepted only if the worst wire length **improves**; otherwise the placement is rolled back and the loop stops.
- Monotone acceptance guarantees the output is never worse than the input.

Design Patterns and Dependencies

- A thin library over the sibling packages `digraphx`, `netlistx`, `physdes`, and `mywheel`.
- **Strategy** for the legalizer, **Memento** for snapshot and rollback, **Adapter** for the solver cost model, **Facade** for the top-level run.
- Bit-identical output on a fixed seed-grid matrix is the **regression oracle**.
- The refactor removed ~2,400 lines of duplicated infrastructure with **zero** result changes. 

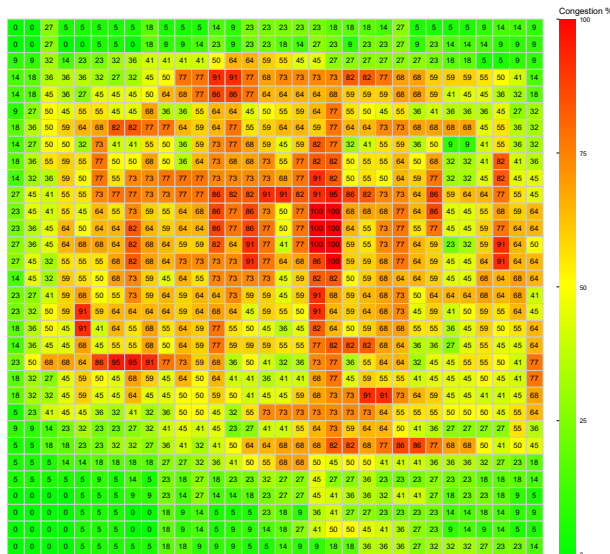
Section 7

Congestion and Results

Visualizing Fairness

Congestion combined (32x32)

busiest cut: 22 wires



Congestion %

100

75

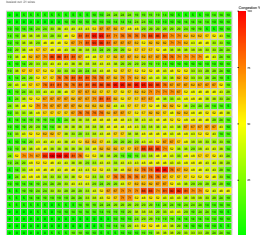
50

25

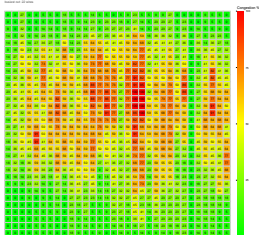
0

x, y and Combined Maps

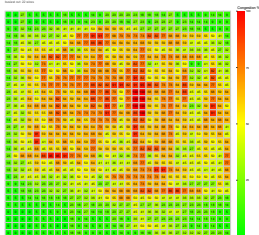
Congestion x-direction (12x12)



Congestion y-direction (12x12)

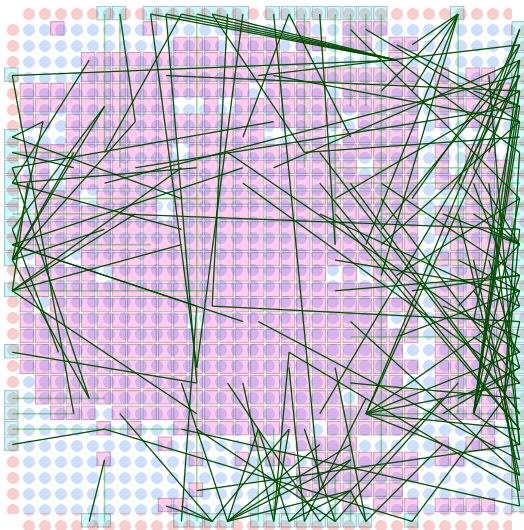


Congestion combined (12x12)



x-direction, y-direction, and their element-wise maximum.

The Placement



Final legal placement on the 32x32 grid: 752 cells, 81 pads, worst wire length 1480

Convergence Across Grid Sizes

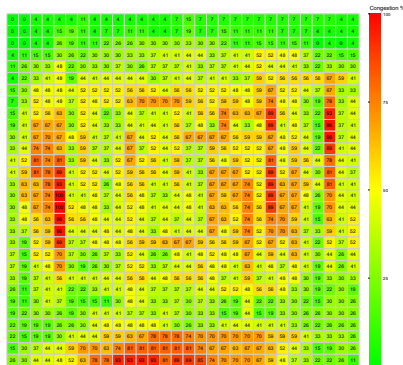
Grid	Slots	Density	Rounds	Worst
30x30	870	86%	2	1640
30x40	1160	65%	2	1440
32x32	992	76%	1	1480
50x50	2450	31%	3	1600
100x100	9900	8%	2	2120

- Dense grids constrain the optimizer; roomy grids are **search**-limited, not capacity-limited.

Density vs Congestion

Congestion combined (30x30)

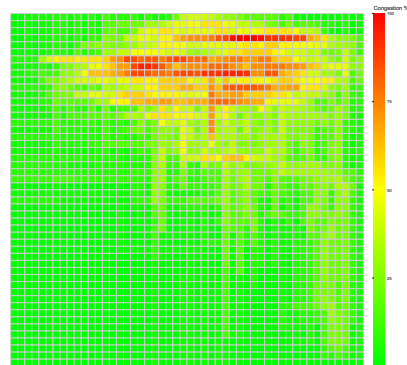
busnet cut: 27 wires



Tight 30x30 (86 percent)

Congestion combined (50x50)

busnet cut: 29 wires



Roomy 50x50 (31 percent)

Computational Cost

Optimization	Effect
Integer floor-division in the cost model	1.85x on the hot callback
Direct linear-assignment solver	2.6x per legalization
Gating infeasible matching calls	7x per matching
Whole seed-grid matrix	4.57x total CPU

- Every change was validated against a **bit-identical** placement oracle.

Preliminary Comparison: NNS vs Aggregate

Benchmark	Placer	Worst	HPWL (M)	Peak cut
p1 (32x32)	NNS	1600	0.57	26
p1 (32x32)	quadratic	2120	0.58	33
ibm01 (120x120)	NNS	8320	50.4	117
ibm01 (120x120)	quadratic	7920	42.1	143

- p1: min-max dominates on every metric.
- ibm01: lower **peak cut**, but higher worst/HPWL at roughly 4× the time.
- That is the **coordinate-descent stall**, not a capacity limit.
- Preliminary only: one seed, a simple quadratic baseline, proxy routing (no sign-off). 

Scaling to Large Designs

- The outer loop is a **block coordinate descent** with monotone acceptance.
- On roomy grids the *search*, not the grid, limits quality (100×100 worst 2120).
- Remedies: **multilevel** (clustering) placement, restarts, a joint two-axis step.
- Exact flow and matching are heavier per node than FFT / linear-system engines.
- Directions: exploit sparsity, warm-start the flow, parallel negative-cycle detection, a GPU port.

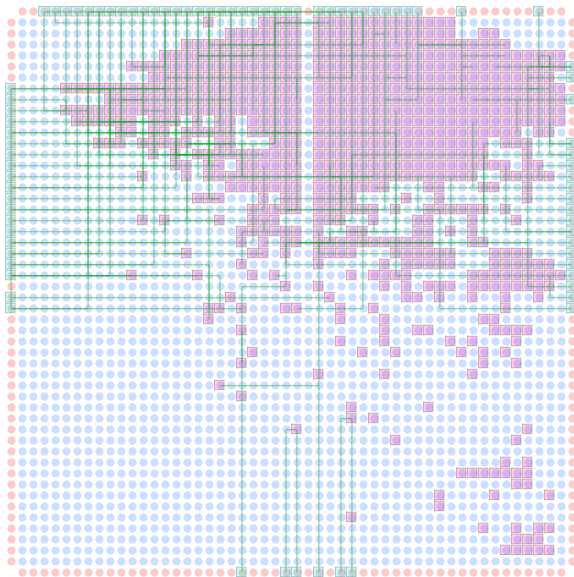
Limitations

- **Benchmark scale:** one synthetic p1; ibm01 only preliminary.
- **No SOTA head-to-head:** a quadratic baseline only; no sign-off router.
- **Narrow metrics:** worst wire length, HPWL, peak cut – no timing or power.
- **Physical abstraction:** 2D uniform grid; no macros, no multi-layer metal.
- **Scalability:** exact flow and matching vs GPU-accelerated analytical engines.

Section 8

Routing and Conclusion

Routed Placement



Summary

- Optimize the **worst** connection, not the total: fairness over aggregation.
- NNS solves it exactly with **Howard's** parametric min-cost flow.
- **Bipartite matching** legalizes; the outer loop is monotone.
- A **weighted** objective extends fairness to timing and power.
- Congestion maps make the fairness/routability trade-off **visible**.
- Preliminary comparison: **lower peak**, but scaling is the open problem. 

Thank You

- Code: <https://github.com/luk036/nnsplace>
- Libraries: digraphx, netlistx, physdes

Questions?



Discussion